SRE: SLO, incident, runbook và phục hồi

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Release là một thay đổi có kiểm soát

Triển khai là hành động đưa phiên bản mới vào môi trường; release là việc cho người dùng sử dụng thay đổi đó. Hai việc có thể tách nhau bằng feature flag hoặc cơ chế điều khiển traffic. Trước mỗi thay đổi, xác định: artifact nào, cấu hình nào, dữ liệu/schema nào, phạm vi tác động, điều kiện thành công và cách quay lại. Một người khác đọc kế hoạch phải biết cách kiểm tra hệ thống sau thay đổi.

Rolling deployment thay dần bản cũ bằng bản mới; blue/green duy trì hai môi trường để chuyển traffic; canary cho một phần nhỏ traffic vào bản mới trước. Mỗi cách đánh đổi tốc độ, tài nguyên, khả năng quan sát và phức tạp. Nếu ứng dụng chỉ có một instance và traffic thấp, một quy trình dừng/ngắn có thể hợp lý hơn xây canary phức tạp. Điều quan trọng là biết rủi ro của lựa chọn.

Rollback ứng dụng không tự động rollback dữ liệu. Nếu bản mới ghi một định dạng database mà bản cũ không hiểu, quay image về sẽ vẫn hỏng. Một chiến lược thường dùng là thay đổi schema theo các bước tương thích: thêm cột/bảng trước, cho mã cũ và mới cùng chạy được, chuyển dữ liệu, sau đó mới bỏ cấu trúc cũ. Bài học đầu tiên: đọc migration cùng với release plan.

Lab release trên môi trường thử

Điều kiện: ứng dụng mẫu đã có ít nhất hai phiên bản và môi trường local/test. Ghi ID artifact của phiên bản A; deploy và gọi health/API. Deploy B với thay đổi nhỏ; gọi cùng request và ghi thời gian, kết quả. Cố ý cấu hình B sai trong môi trường thử, quan sát lỗi, quay về A; sau đó kiểm tra dữ liệu còn nguyên. Viết checklist gồm điều kiện dừng/rollback, người thực hiện và bằng chứng.

SLI, SLO và alert đáng hành động

SLI là chỉ số thể hiện chất lượng dịch vụ theo góc nhìn người dùng, ví dụ tỷ lệ request hợp lệ được trả thành công hoặc độ trễ dưới ngưỡng. SLO là mục tiêu cho SLI trong một khoảng thời gian. Error budget là phần sai lệch được phép khi so với mục tiêu. Ví dụ minh họa: nếu SLO là 99,9% request hợp lệ thành công trong 30 ngày, budget lý thuyết là 0,1% request không đạt; cách tính thực tế phải định nghĩa rõ request nào thuộc mẫu và cách xử lý lưu lượng thấp.

Không phải metric nào cũng nên gọi người trực. Alert tốt chỉ tới vấn đề có tác động đáng kể và có hành động khả thi. Nếu mỗi lần CPU tăng là một cuộc gọi nhưng dịch vụ vẫn ổn, nhóm sẽ mệt và bỏ qua cảnh báo quan trọng. Với hệ thống thực, cảnh báo có thể dựa trên tốc độ đốt error budget; cần thử ngưỡng và giảm nhiễu theo dữ liệu thật.

Bài tập định nghĩa SLO

Cho API ghi chú, hãy viết: ai là người dùng, request nào hợp lệ, "thành công" là status nào, thời gian đo là bao lâu và dữ liệu lấy từ đâu. Sau đó nêu hai giới hạn của chỉ số: có thể bỏ sót trải nghiệm nào? Nếu database không có request trong 10 phút, liệu 100% thành công có chứng minh dịch vụ tốt không? Đây là bài tập tư duy trước khi chọn Prometheus hoặc công cụ khác.

Incident, runbook và postmortem

Khi có sự cố, ưu tiên hiểu tác động và khôi phục an toàn. Ghi thời điểm bắt đầu, triệu chứng, phạm vi người dùng, thay đổi gần đây và giả thuyết. Nếu bản deploy mới gây lỗi, rollback có thể nhanh hơn điều tra sâu trong lúc người dùng đang bị ảnh hưởng. Sau khi khôi phục, giữ bằng chứng để tìm nguyên nhân gốc và cải thiện quy trình. Một người phụ trách điều phối có thể giúp tránh nhiều người cùng đổi cấu hình không ghi nhận.

Runbook là hướng dẫn thao tác cho tình huống lặp lại: xác minh triệu chứng, nguồn dữ liệu, bước giảm tác động, điểm dừng và đường liên hệ. Postmortem là tài liệu học từ sự cố: timeline, tác động, yếu tố góp phần, điều đã hiệu quả, điều cần cải thiện và người theo dõi hành động. Mục tiêu là sửa hệ thống và quy trình, không quy lỗi cá nhân.

Backup phải được thử restore trong môi trường an toàn. RPO là lượng dữ liệu tối đa chấp nhận mất (tính theo thời gian); RTO là thời gian mục tiêu để dịch vụ hồi phục. Hai mục tiêu này chi phối tần suất backup và kiến trúc. "Có backup" mà không biết restore mất bao lâu chưa đủ để cam kết phục hồi.

Diễn tập sự cố

Trong môi trường thử, làm ứng dụng trả lỗi bằng cấu hình sai. Ghi phút phát hiện, tín hiệu quan sát, giả thuyết, hành động giảm ảnh hưởng, thời điểm khôi phục. Viết postmortem một trang. Không diễn tập trên hệ thống có người dùng thật khi chưa có kế hoạch và quyền phù hợp.

Tính SLI và error budget bằng ví dụ cụ thể

Giả sử trong 30 ngày API nhận 100.000 request hợp lệ. Có 99.850 request được trả thành công theo định nghĩa nhóm đã thống nhất. Availability SLI = 99.850 / 100.000 = 99,85%. Nếu SLO là 99,9%, error budget cho 100.000 request là 100 request không đạt; thực tế có 150 request không đạt, tức đã vượt budget 50. Trong dịch vụ thật, số request thay đổi liên tục nên hệ thống tính theo cửa sổ thời gian và quy tắc lọc rõ ràng. Ví dụ này chỉ giúp hiểu phép tính.

Định nghĩa "thành công" cần thận trọng. Một HTTP 200 có thể trả dữ liệu sai; một HTTP 404 do người dùng yêu cầu ID không tồn tại có thể là hành vi đúng, không nên tính là lỗi hệ thống. Nếu client hủy request, có tính vào mẫu không? Nếu chỉ đo từ server, lỗi DNS trước khi tới server sẽ không xuất hiện. Hãy ghi phạm vi chỉ số và giới hạn của nguồn dữ liệu.

Latency SLI có thể là tỷ lệ request dưới một ngưỡng thay vì chỉ lấy trung bình. Ví dụ 99 request nhanh và 1 request rất chậm có thể tạo trung bình trông ổn nhưng vẫn có người dùng trải nghiệm kém. Tập nhìn phân bố, percentile và số lượng mẫu; percentile trên ít request có thể dao động mạnh. Đừng đặt mục tiêu bằng con số tùy hứng: cần nhu cầu người dùng và dữ liệu thực tế.

Bài tập tính và diễn giải

Trong 10.000 request, có 9.990 request được xem là thành công. Nếu SLO là 99,95%, budget là 5 lỗi; thực tế có 10 lỗi, vượt budget 5. Hãy tự tính trước khi đọc câu giải. Nếu cả 10 lỗi xảy ra trong một phút, trải nghiệm người dùng khác với 10 lỗi rải đều; vì vậy alert còn cần xem tốc độ lỗi, không chỉ tổng tháng.

Viết runbook cho một lỗi cụ thể

Ví dụ: sau deploy, tỷ lệ HTTP 5xx của API tăng. Runbook nên bắt đầu bằng xác nhận ảnh hưởng: dashboard nào, bao nhiêu request, môi trường nào, bắt đầu từ lúc nào. Sau đó kiểm tra phiên bản vừa deploy, log lỗi, health/readiness, kết nối database và dependency. Nếu lỗi gắn chặt với bản mới, ghi điều kiện rollback và cách kiểm tra sau rollback. Nếu không, tránh đổ lỗi cho deploy chỉ vì chúng gần nhau về thời gian.

Runbook cần có "điểm dừng": khi nào không tự thử tiếp và phải báo người có quyền/kinh nghiệm hơn. Ví dụ dữ liệu có nguy cơ hỏng, cần chạy migration ngược, hoặc chỉ số không hồi phục sau rollback. Nó cũng cần ghi các lệnh là chỉ đọc hay thay đổi, môi trường nào được phép chạy và cách lưu bằng chứng. Runbook cũ có thể nguy hiểm; trước khi dùng trong production, xác nhận phiên bản và quyền hiện tại.

Mẫu runbook rút gọn

```
Trieu chung: 5xx vuot nguong sau release
Xac nhan: thoi gian bat dau, service, moi truong, pham vi nguoi dung
Thu thap: version, release event, logs, readiness, database connectivity
Giam tac dong: rollback neu bang chung cho thay version moi gay loi
Xac minh: smoke request, 5xx trend, latency, data integrity
Dung va bao: neu can can thiep du lieu hoac rollback khong phuc hoi
Sau su co: timeline, root cause, action co owner va han xem lai
```

Phân biệt backup, restore và disaster recovery

Backup là bản dữ liệu; restore là thao tác đưa dữ liệu từ bản đó trở lại một hệ thống có thể dùng được. Disaster recovery là kế hoạch rộng hơn: người, quyền, hạ tầng, ứng dụng, dữ liệu, DNS và kiểm tra. Trong bài lab, hãy đo thời gian khôi phục cả API chứ không chỉ thời gian copy file database. Nếu file đã được phục hồi nhưng API vẫn dùng sai đường dẫn, người dùng chưa nhận được dịch vụ.

Chọn một sự cố giả: máy chạy ứng dụng mất hoàn toàn. Liệt kê thứ cần tái tạo theo thứ tự: môi trường, cấu hình/secret, artifact ứng dụng, database, route/DNS, kiểm tra. Xác định phần nào có trong Git/IaC, phần nào được backup, phần nào đang nằm trong trí nhớ một người. Chỗ cuối là điểm yếu cần ghi lại. Lặp diễn tập với dữ liệu giả cho tới khi tài liệu đủ để người khác thực hiện.

Reliability, failure mode và phục hồi dữ liệu

Reliability là khả năng hệ thống thực hiện đúng chức năng khi người dùng cần. Một thiết kế đáng tin cậy bắt đầu bằng xác định service level mong muốn, failure modes và chi phí chấp nhận. Lỗi có thể là process crash, VM mất, database chậm, đĩa đầy, credential hết hạn, deploy sai hoặc lỗi người vận hành. Với mỗi lỗi, hỏi: tín hiệu nào phát hiện, phạm vi ảnh hưởng, thao tác giảm tác động và cách ngăn lặp lại.

Redundancy giúp vượt qua một số lỗi nhưng không tự động giải quyết lỗi cấu hình chung, xóa dữ liệu hoặc dependency chung. Autoscaling giúp khi tải tăng trong giới hạn, nhưng có thể làm quá tải database nếu không xem toàn chuỗi. Circuit breaker, timeout và retry có thể giúp chịu lỗi tạm thời; retry thiếu giới hạn có thể khuếch đại sự cố. Trước khi thêm cơ chế, mô tả rõ failure mode cần xử lý.

Backup, replication và snapshot có mục đích khác nhau. Backup là bản dữ liệu để phục hồi tại một thời điểm; replication giữ nhiều bản sao gần như liên tục; snapshot là cơ chế chụp trạng thái của một tài nguyên, cần hiểu tính nhất quán. Chính sách backup phải nêu tần suất, thời gian giữ, quyền, mã hóa và kiểm tra restore. RPO/RTO phải có căn cứ từ nhu cầu của ứng dụng. Nếu mất tối đa một giờ dữ liệu là chấp nhận được, một backup mỗi ngày sẽ không đáp ứng khi xảy ra sự cố gần cuối ngày.

Lab restore thực tế

Tạo database thử với vài ghi chú giả. Ghi số bản ghi và một giá trị kiểm tra. Tạo backup theo tài liệu database bạn chọn, xóa hoặc thay đổi dữ liệu trong bản thử, restore vào một database thử khác, rồi so sánh số bản ghi và nội dung. Đo thời gian từ lúc bắt đầu restore đến khi API dùng được. Không thử trên dữ liệu thật. Nếu bản backup không mở được, ghi rõ lỗi thay vì đánh dấu mục tiêu đã đạt.

Incident và quyết định kỹ thuật

Trong sự cố, người giỏi không đoán nhanh nhất mà giữ được chuỗi quan sát và thay đổi kiểm soát: xác nhận tác động, kiểm tra thay đổi gần đây, khoanh vùng, giảm ảnh hưởng, ghi timeline rồi phân tích sâu sau khi ổn định. Chia vai nếu nhiều người cùng xử lý: một người điều phối, một người chẩn đoán, một người cập nhật tình hình. Quy trình này đặc biệt quan trọng khi quyền thay đổi production cao.

Một quyết định kiến trúc cần ghi mục tiêu, bối cảnh, các lựa chọn hợp lý, lợi/hại, chi phí và điều kiện xem xét lại. Ví dụ: chọn database managed thay vì tự chạy trên VM vì nhóm nhỏ muốn giảm công việc backup/patching, đổi lại chi phí tiền cao hơn và phụ thuộc dịch vụ. Lập luận có dữ liệu và giới hạn sẽ có giá trị trong phỏng vấn hơn việc chỉ liệt kê công cụ.

Diễn tập quyết định khi hệ thống lỗi

Giả sử sau deploy phiên bản B, 30% request tạo ghi chú trả 500; đọc ghi chú vẫn hoạt động. Một cách xử lý có trình tự: xác nhận dashboard/log và thời điểm; xem B thay đổi mã hay schema nào; thử một request có request ID; so sánh log B với A; cân nhắc rollback B nếu tương thích dữ liệu; kiểm tra tỷ lệ lỗi giảm và bản ghi không bị mất; cuối cùng phân tích lỗi và bổ sung test. Nếu migration của B đã đổi schema không tương thích, rollback image ngay có thể tệ hơn. Hãy ghi điểm chưa biết và dừng thay đổi dữ liệu cho tới khi hiểu.

Trong phỏng vấn, người đánh giá có thể cố tình không cho đủ dữ kiện. Câu trả lời tốt nêu dữ kiện cần thu thập và quyết định phụ thuộc vào chúng. Một câu "chỉ cần restart" không đủ vì restart có thể xóa triệu chứng tạm thời mà làm mất log/bằng chứng. Cũng không phải lúc nào root cause phải tìm trước khi khôi phục; hãy giảm ảnh hưởng trước nếu có hành động an toàn.

Reliability là một chuỗi phụ thuộc

Một API có thể có ba replica nhưng vẫn phụ thuộc vào một database, một DNS service, một secret và một đường mạng. Vẽ dependency graph rồi hỏi mỗi cạnh: nếu phụ thuộc chậm, timeout ở đâu; nếu lỗi, có fallback không; retry có làm tải tăng không; có tín hiệu nào phát hiện không. Khi một request đi qua ba service mà mỗi service có xác suất thành công thấp hơn 100%, tổng khả năng thành công có thể thấp hơn từng phần. Không cần tính con số chính xác để thấy rằng thêm tầng có thể thêm failure mode.

Ví dụ người dùng nhấn Lưu, API gọi DB nhưng timeout. Nếu API retry ngay ba lần cho mọi request, DB đang quá tải có thể nhận gấp nhiều lần số truy vấn. Cách xử lý có thể là timeout hợp lý, retry giới hạn với backoff và chỉ retry thao tác an toàn/idempotent theo thiết kế. Nếu POST tạo ghi chú bị retry sau khi DB đã ghi nhưng response thất lạc, người dùng có thể có hai ghi chú. Idempotency key là một cách giải quyết khi nghiệp vụ cần; lab hiện tại chưa có, nên ghi rõ giới hạn.

Bài tập failure mode

Liệt kê năm phụ thuộc của Notes API: process, file/volume, CPU/memory, network, client hoặc database service tương lai. Với mỗi phụ thuộc, ghi triệu chứng, tín hiệu, cách giảm tác động và cách khôi phục. Ví dụ volume đầy -> POST lỗi/SQLite ghi thất bại -> metric dung lượng + log -> ngừng ghi không cần thiết/giải phóng theo quy trình -> tăng dung lượng hoặc restore nếu data bị hỏng. Không tùy tiện xóa file database để lấy chỗ trống.

Reliability là lời hứa đo được

“Hệ thống HA” không đủ để điều hành. Hãy xác định dịch vụ nào, người dùng nào, SLI nào và cửa sổ nào. Một API có ba Pod nhưng DB một điểm hỏng có thể vẫn vi phạm SLO. SRE dùng SLO/error budget để thảo luận rủi ro release: khi còn nhiều budget, đội có thể chấp nhận thử nghiệm có guardrail; khi burn nhanh, ưu tiên ổn định và giảm tác động. Đây là chính sách ra quyết định, không phải con số để phạt đội.

RTO là thời gian phục hồi mục tiêu sau thảm họa; RPO là mức dữ liệu có thể mất tối đa tính theo thời gian. Snapshot mỗi ngày có thể tạo RPO tới gần 24 giờ nếu không có log/replication bổ sung. Có replica không đồng nghĩa có backup: lỗi xóa dữ liệu hoặc ransomware có thể lan sang replica. Có backup không đồng nghĩa phục hồi trong RTO: cần đo thời gian lấy bản sao, khôi phục, kiểm tra dữ liệu, chuyển DNS/traffic và mở lại dịch vụ. “Restore thành công” phải bao gồm kiểm tra ứng dụng đọc/ghi dữ liệu đúng.

Một incident từ phút 0 đến postmortem

Tình huống: lúc 10:02 alert 5xx của Notes API tăng lên 8%; deploy mới lúc 09:58. Người trực ca xác nhận tác động bằng synthetic request và dashboard, lập incident, ghi timeline. Chỉ định incident commander điều phối, người điều tra kỹ thuật, người truyền thông khi quy mô đủ lớn. Người điều tra tìm bằng chứng: revision, route lỗi, DB status, log/trace; người điều phối quyết định giảm tác động. Không để cả đội cùng sửa production mà không ai ghi thay đổi.

10:07: giả thuyết migration mới làm query lỗi. Nếu có bằng chứng rõ và rollback app còn tương thích schema, rollback theo runbook; nếu không, tạm tắt feature/route hoặc roll-forward hotfix. **10:15:** theo dõi 5xx, p95, backlog và request thành công thực, không đóng incident ngay khi Pod xanh. **10:30:** cập nhật trạng thái cho stakeholder bằng dữ kiện: ảnh hưởng, hành động, thời gian cập nhật tiếp theo; tránh hứa ETA không có cơ sở. Sau phục hồi, viết postmortem không đổ lỗi cá nhân: timeline, nguyên nhân gần/gốc, yếu tố phát hiện muộn, điều gì đã làm tốt, action có owner/deadline/kiểm chứng.

Runbook ngắn cho lỗi DB connection exhaustion

Trigger: API 5xx và timeout tăng; DB connection gần trần. **Xác nhận:** dashboard theo route, pool active/idle, DB connection count, query slow/lock, revision gần nhất. **Giảm tác động:** dừng rollout mới, giảm traffic tính năng gây bão hoặc rollback nếu tương thích; tránh tăng max connections vô hạn vì DB có giới hạn bộ nhớ. **Phục hồi:** kiểm tra request mới, p95, connection và queue sau hành động. **Leo thang:** gọi DB owner khi nghi replication/lock; lưu log/trace trước khi chúng hết retention. **Sau sự cố:** test load, pool timeout/retry budget, circuit breaker và alert sớm. Runbook phải được tập, không chỉ viết.

Bài tập tính toán để không nhầm SLO

Trong 30 ngày có 2.000.000 request hợp lệ, SLO good request 99,95%. Budget là 0,05% x 2.000.000 = 1.000 request không tốt. Nếu một sự cố 12 phút gây 700 request lỗi, còn 300 request budget theo mô hình đơn giản; không thể nói “12 phút chỉ chiếm ít thời gian” để bỏ qua. Nếu traffic tăng gấp đôi, số request lỗi cho phép theo tỷ lệ cũng đổi. Nếu SLI đo availability bằng phút, phép tính khác; luôn nêu đơn vị và mẫu số.

Một burn rate 10x nghĩa tiêu budget nhanh gấp 10 so với tốc độ cho phép trung bình, không phải cứ 10% traffic lỗi. Thiết kế nhiều cửa sổ để nhận cả lỗi lớn nhanh và lỗi nhỏ kéo dài; không page vì một điểm dữ liệu đơn lẻ khi hệ thống nhiễu. Google SRE Workbook có hướng dẫn alert trên SLO và incident response để đào sâu.

Failure domain và diễn tập

“Ba VM, ba replica” cần hỏi ba VM ở host/rack/zone nào; storage, DB, load balancer, identity và DNS có chung điểm hỏng không. Chọn một lỗi cụ thể để game day trên môi trường thử: mất một node, mất DB primary, DNS private trả IP sai hoặc registry tạm không truy cập được. Trước khi diễn tập, xác định phạm vi, cách dừng, tín hiệu dự kiến và người quan sát. Trong diễn tập, ghi thời gian phát hiện, quyết định, hành động, phục hồi và khoảng trống runbook. Không chủ động phá production chỉ để “thử HA” khi chưa có kế hoạch/ủy quyền.

Câu hỏi có lời giải: backup hằng ngày có RPO 1 giờ không? Không nếu không có cơ chế bổ sung và restore chứng minh được. **HPA có làm hệ thống HA?** Nó tăng Pod theo metric nhưng không sửa DB hay failure domain chung. **Postmortem có kết luận “người triển khai quên kiểm tra”?** Chưa đủ; cần chỉ ra vì sao gate, review, thiết kế và tín hiệu cho phép lỗi lọt qua, rồi đặt action kiểm chứng được.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://sre.google/workbook/alerting-on-slos/

- https://sre.google/workbook/incident-response/